

UNIVERSIDAD EL BOSQUE

Facultad de Ingeniería — Programa de Ingeniería de Sistemas

Bases de Datos 1 (12998) — Periodo 2026-2

Conexión de la Aplicación Web a la Base de Datos Relacional

SIGEVET — Sistema Integral de Gestión Veterinaria

Red de Clínicas Veterinarias Huella Sana S.A.S. (caso simulado)

Proyecto Final · Fase 2 · Versión 2.0

Integrantes

Sebastián Castellanos Parra

Juan David Polo Herrera

Juan Miguel Ramos Sevilla

Sergio Alejandro Parra Rivera

Docente

Ing., Esp., M.Ed. Christian Felipe Duarte Arévalo

Bogotá D.C.

Tabla de contenido

1 Introducción

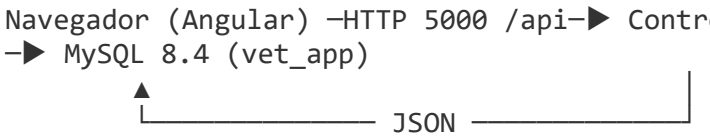
La aplicación web trabaja sobre la base `sigevet` de la VM: gestiona clientes, mascotas con su historia clínica, citas, facturas, inventario y personal, y ejecuta en vivo los **55 ejercicios de álgebra relacional**. Cada cambio pasa por las restricciones y los disparadores de la base.

2 Arquitectura

Dos partes en un solo `sigevet.jar`:

- **Backend en Java 21 y Spring Boot 3.5**: API REST (`@RestController`, rutas `/api/...`) que responde en JSON. Accede a la base con JDBC, SQL escrito a mano y `PreparedStatement`, **sin ORM**.
- **Frontend en Angular 21** con Bootstrap 5: pide los datos a la API con `HttpClient` y nunca se conecta a MySQL.

`ng build` genera el frontend y Maven lo copia a `static/` dentro del jar. Spring Boot sirve páginas y API en el puerto 5000, como el servicio `sigevet-web` de la VM.



Capas del backend: el controlador recibe la petición, el servicio valida y aplica las reglas, y el DAO ejecuta el SQL.

Organización del backend

Paquete (app/src/main/java/co/edu/unbosque/sigevet/)	Qué tiene
`connection/`	`ConexionBD` abre la conexión con `DriverManager`; `Transaccion` hace ROLLBACK y cierra
`model/`	Clases con atributos privados, getters y setters: `Persona`, `Cliente`, `Empleado`, `Veterinario`, `Auxiliar`, `Mascota`, `Cita`, `Factura`, etc., y los enums de estados
`dto/`	Respuestas (`ClienteDetalleDTO`, `FacturaDetalleDTO`) y formularios de POST/PUT (`ClienteFormDTO`, `FacturaFormDTO`)
`dao/`	Una clase por tema (`ClienteDAO`, `MascotaDAO`, `CitaDAO`, `FacturaDAO`, `InventarioDAO`, `PersonalDAO`…) con el SQL
`service/`	Un `...Service` por tema: valida, aplica las reglas y coordina los DAO
`controller/api/`	Un `...ApiController` por recurso (inicio, clientes, mascotas, citas, facturas, inventario, personal, catálogos y álgebra)
`exception/`	`RecursoNoEncontradoException` (404),

Paquete (app/src/main/java/co/edu/unbosque/sigevet/)	Qué tiene
	DatosInvalidosException (400), ConflictoDatosException (409) y ManejadorErroresApi, que las convierte en JSON {estado, mensaje}
util/	Validador (reglas de los formularios), ErroresSQL (traduce los errores de MySQL) y TextoUtil
config/	ConfiguracionWeb: sirve el frontend y devuelve index.html en las rutas de Angular (recargar /clientes/5 funciona)
algebra/	Lector de los 55 ejercicios de 01_consultas_algebra_relacional.sql

Organización del frontend

Carpeta (frontend/src/app/)	Qué tiene
models/models.ts	Interfaces TypeScript con los campos del JSON
services/api.service.ts	Un servicio por recurso (HttpClient → Observable)
components/	Un componente por página, en carpetas con sufijo component: login, inicio, clientes, mascotas, agenda, inventario, facturas, personal, álgebra, sus detalles y los formularios de cliente, mascota, cita, factura y empleado, más el menú (navbarcomponent) y el pie de página (footercomponent)
app-module.ts	Módulo raíz (NgModule): declara todos los componentes y registra HttpClient, el interceptor de sesión y el idioma es-CO
app-routing-module.ts	Rutas del navegador (RouterModule.forRoot); todas, menos /login, pasan por sesionGuard
guards/	sesionGuard (exige sesión) e interceptor que lleva a /login cuando la API responde 401

3 Programación orientada a objetos: herencia y polimorfismo

La jerarquía reproduce la especialización EMPLEADO–VETERINARIO/AUXILIAR del EER:

```
Persona (abstracta: nombres, apellidos, documento, email; getDescripcion()
abstracto)
├── Cliente
└── Empleado (sede, tipo, salario, ...)
    ├── Veterinario (tarjeta profesional, universidad, especialidades)
    └── Auxiliar (área, turno)
```

PersonalDAO une EMPLEADO con sus subtipos y crea Veterinario, Auxiliar o Empleado según tipo_empleado, en un ArrayList<Empleado>. Cada elemento trae los campos de su clase real y el

resultado de su propio `getDescripcion()` (`@Override`), que muestra la página de personal. Eso es **polimorfismo**: Java ejecuta la versión del objeto concreto aunque la lista sea de empleados.

```
Empleado empleado;
String tipoEmpleado = rs.getString("tipo_empleado");
if ("VET".equals(tipoEmpleado)) {
    empleado = new Veterinario();
} else if ("AUX".equals(tipoEmpleado)) {
    empleado = new Auxiliar();
} else {
    empleado = new Empleado();
}
```

4 Conexión a MySQL

MySQL Connector/J conecta por JDBC. `ConexionBD` abre una conexión por operación y el DAO la cierra con `try-with-resources`:

```
public Connection conectar() throws SQLException {
    return DriverManager.getConnection(url, usuario, clave);
}
```

Los DAO asignan los parámetros `?` con `PreparedStatement` (`sentencia.setInt(1, id)`) y leen el `ResultSet` dentro del `try-with-resources`.

Configuración de la conexión

Dato	Valor usado en el proyecto	Variable de entorno
Usuario	`vet_app` (lectura de toda la base, escritura solo en las tablas de la app)	`SIGEVET_DB_USER`
Clave	No se escribe en el código	`SIGEVET_DB_PASSWORD`
URL JDBC	`jdbc:mysql://localhost:3306/sigevet` (misma VM)	`SIGEVET_DB_URL`

`vm/scripts/instalar_app.sh` guarda estos valores en `/opt/sigevet/app.env`, accesible solo al dueño y leído por `sigevet-web`.

5 API REST

Cada controlador llama a su servicio y devuelve un objeto que Spring convierte en JSON. Ejemplo de `ClienteApiController`:

```
@GetMapping("/{id}")
public ClienteDetalleDTO detalle(@PathVariable int id) {
    return clienteService.detalle(id);
}
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public IdDTO crear(@RequestBody ClienteFormDTO form) {
```

```
    return new IdDTO(clienteService.crear(form));
}
```

Recursos de la API

Recurso	Lectura	Escritura
Sesión	`GET /api/sesion`	POST (entrar), DELETE (salir)
Inicio	`GET /api/inicio`	—
Clientes	`GET /api/clientes`, `GET /api/clientes/{id}`	POST, PUT `/{id}`, DELETE `/{id}` (solo sin historial)
Mascotas	`GET /api/mascotas`, `GET /api/mascotas/{id}`	POST, PUT `/{id}`, PATCH `/{id}/estado`
Citas	`GET /api/citas?sede=&estado=`	POST, PATCH `/{id}/estado`
Facturas	`GET /api/facturas`, `GET /api/facturas/{id}`	POST (con sus líneas), PATCH `/{id}/estado`
Inventario	`GET /api/inventario?sede=`	POST, PUT, DELETE
Personal	`GET /api/personal?sede=`	POST, PUT `/{id}`, PATCH `/{id}/estado`
Álgebra	`GET /api/algebra`, `GET /api/algebra/{numero}`	—
Catálogos	`GET /api/catalogos/...`	—

Mascotas, empleados, citas y facturas cambian de estado en vez de borrarse, porque deben conservarse.

6 Transacciones con JDBC

Cuando una operación escribe en varias tablas, el DAO abre la transacción a mano. Al crear una factura se insertan el encabezado y las líneas y, si se vende un medicamento, se descuenta el inventario de la sede. Si un paso falla (por ejemplo, no hay existencias), ROLLBACK deshace todo:

```
conexion = conexionBD.conectar();
conexion.setAutoCommit(false);
// 1. consecutivo FV-... (SELECT ... FOR UPDATE)
// 2. INSERT INTO FACTURA ...
// 3. por línea: INSERT INTO DETALLE_FACTURA ... y, si es medicamento,
//    UPDATE INVENTARIO SET stock_actual = stock_actual - ?
//    WHERE ... AND stock_actual >= ?
conexion.commit();
...
} catch (SQLException e) {
    Transaccion.deshacer(conexion);
    throw ErroresSQL.traducir(e);
} finally {
    Transaccion.cerrar(conexion);
}
```

El mismo patrón se usa al crear un empleado con su subtipo y al anular una factura, que devuelve los medicamentos al inventario.

7 Validaciones y reglas de la base

Tres niveles de validación:

1. **Formulario en Angular**: campos obligatorios, longitudes y formatos.
2. **Servicio en Java** (`Validador`): documentos de 6 a 10 dígitos, nombres solo con letras, teléfonos de 7 a 10 dígitos, microchip de 15 dígitos, fechas y rangos. Si falla, responde 400.
3. **Base de datos**: CHECK, UNIQUE, llaves foráneas y disparadores. La app no los repite en Java; si MySQL rechaza un dato, `ErroresSQL` traduce el error y la página lo muestra.

Traducción de los errores de la base

Error de MySQL	Respuesta de la API
`SQLSTATE 45000` (SIGNAL de un disparador)	409 “Regla de negocio: …” con el mensaje de la base
1062 (valor repetido en PK o UNIQUE)	409 “Ya existe un cliente con ese tipo y número de documento.”
3819 (CHECK)	409 con el nombre de la restricción que se incumplió
1451 / 1452 (llave foránea)	409 “tiene registros relacionados” o “el dato seleccionado no existe”

8 Inicio de sesión

Solo el personal usa la aplicación. Entra con su **correo corporativo** y su **número de documento**; no hay tabla de usuarios aparte porque esos datos ya están en EMPLEADO (el correo es UNIQUE). `SesionDAO` consulta con parámetros:

```
String sql = "SELECT e.id_empleado, e.nombres, e.apellidos, e.tipo_empleado "
        + "FROM EMPLEADO e WHERE e.email = ? AND e.num_documento = ? AND
e.estado = 'ACTIVO'";
```

Si hay una fila, `SesionApiController` guarda el empleado en la sesión HTTP (cookie `JSESSIONID`) y Angular muestra su nombre con el botón **Salir**. Un empleado `INACTIVO` no entra. `FiltroSesion` revisa cada petición a `/api/...` y, sin sesión, responde 401; en Angular, `sesionGuard` y un interceptor llevan a `/login`.

Es un inicio de sesión de demostración: el documento hace de contraseña y no se guardan contraseñas cifradas, fuera del alcance (Propuesta del Proyecto).

9 Seguridad

- `vet_app` tiene SELECT sobre toda la base (`rol_sigevet_lectura`) y escritura solo en las tablas que modifica la app (`rol_sigevet_escritura`, `database/01_ddl/05_permisos_app.sql`): sin DROP, ALTER ni CREATE, sin cambios en catálogos y con DELETE solo en teléfonos, inventario, especialidades y clientes sin historial.
- La API solo responde a un empleado activo con sesión (`FiltroSesion`, 401).
- `PreparedStatement` con `?` en todo el SQL evita la inyección; Angular muestra los datos como texto (`{{ }}`), que escapa el HTML.
- La ruta de álgebra elige uno de los 55 ejercicios por número; no recibe SQL.

- La clave de la base va en una variable de entorno, no en el código ni en GitHub.

10 Ejecución

En la VM (forma recomendada), desde la raíz del proyecto:

```
bash vm/scripts/cargar_bd.sh --reiniciar   # base, datos y permisos de
vet_app
sudo bash vm/scripts/instalar_app.sh       # Java 21 y Node.js 22, compila y
activa el servicio
systemctl status sigevet-web
```

En cualquier equipo con Java 21 y Node.js 22:

```
cd frontend && npm ci && npx ng build      # compila el frontend
cd ../app && ./mvnw package                # arma sigevet.jar con el frontend
export SIGEVET_DB_PASSWORD='<clave de vet_app>'
java -jar target/sigevet.jar               # http://localhost:5000
```

11 Evidencias de funcionamiento

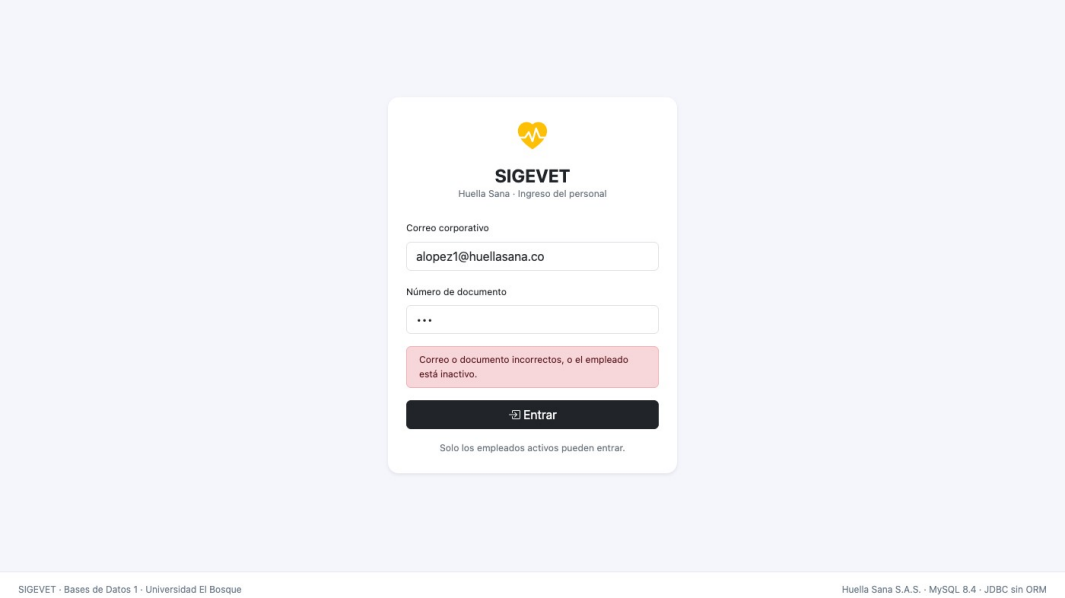
La aplicación (frontend en Angular y API REST en Spring Boot, sin ORM) se ejecutó como el servicio `sigevet-web` dentro de la VM, conectada a MySQL con el usuario `vet_app`, y se abrió desde el navegador del equipo anfitrión en el puerto 5000. Las pruebas se hicieron en la VM de simulación, que tiene el mismo código, la misma base y los mismos scripts que el servidor Fedora.

Pruebas de la aplicación contra MySQL

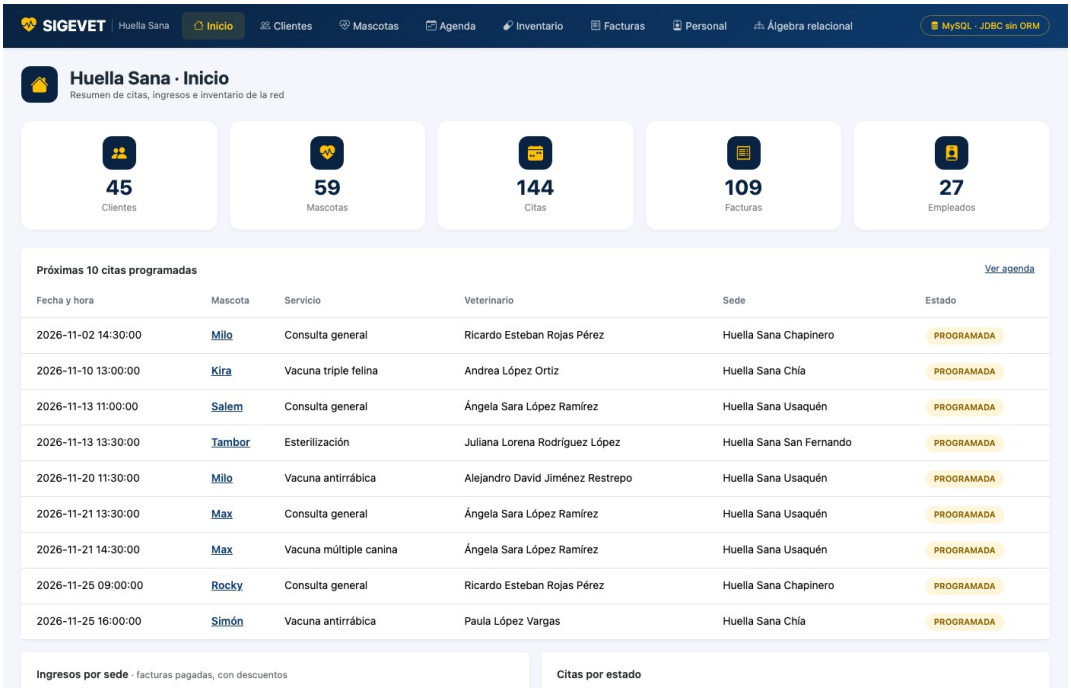
Prueba	Respuesta	Resultado
`GET /api/clientes` sin iniciar sesión	HTTP 401	"Inicie sesión para usar la aplicación." La API no ejecuta la consulta
`POST /api/sesion` con un documento que no corresponde	HTTP 401	"Correo o documento incorrectos, o el empleado está inactivo."
`POST /api/sesion` con `x' OR '1'='1` en los dos campos	HTTP 401	La inyección SQL no funciona: los valores van como parámetros ?
`POST /api/sesion` con el correo y el documento de un veterinario activo	HTTP 200	Sesión iniciada; el menú muestra su nombre y el botón Salir
`DELETE /api/sesion` y luego `GET /api/clientes`	HTTP 204 y 401	Al salir, la API vuelve a pedir inicio de sesión
`GET /api/inicio` y las páginas de consulta (con sesión)	HTTP 200	Indicadores, clientes, mascotas con historia clínica, agenda, inventario, facturas, personal y álgebra con datos de la base
`GET /api/algebra/1` a `/api/algebra/55`	HTTP 200	Los 55 ejercicios de álgebra ejecutados en MySQL
`POST /api/clientes` (cliente nuevo con un teléfono)	HTTP 201	Se creó el cliente 46 con su teléfono
`POST /api/clientes` con el	HTTP 409	La base lo rechazó (UNIQUE): "Ya

Prueba	Respuesta	Resultado
mismo tipo y número de documento		existe un cliente con ese tipo y número de documento.”
POST `/api/clientes` con letras en el documento	HTTP 400	El servicio lo rechazó antes de llegar a la base: “solo números, de 6 a 10 dígitos”
POST `/api/facturas` con un servicio y 5 unidades de Amoxivet en la sede Usaquén	HTTP 201	Factura 110 creada; las existencias de Amoxivet en la sede bajaron de 88 a 83
POST `/api/facturas` con 1.000 unidades (hay 83)	HTTP 409	“No hay existencias suficientes de Amoxivet Tableta 500 mg: hay 83 y se pidieron 1000.” Se hizo ROLLBACK: siguen 110 facturas y 83 unidades
GET `/api/clientes/99999`	HTTP 404	`{"estado":404,"mensaje":"No existe el cliente 99999"}`

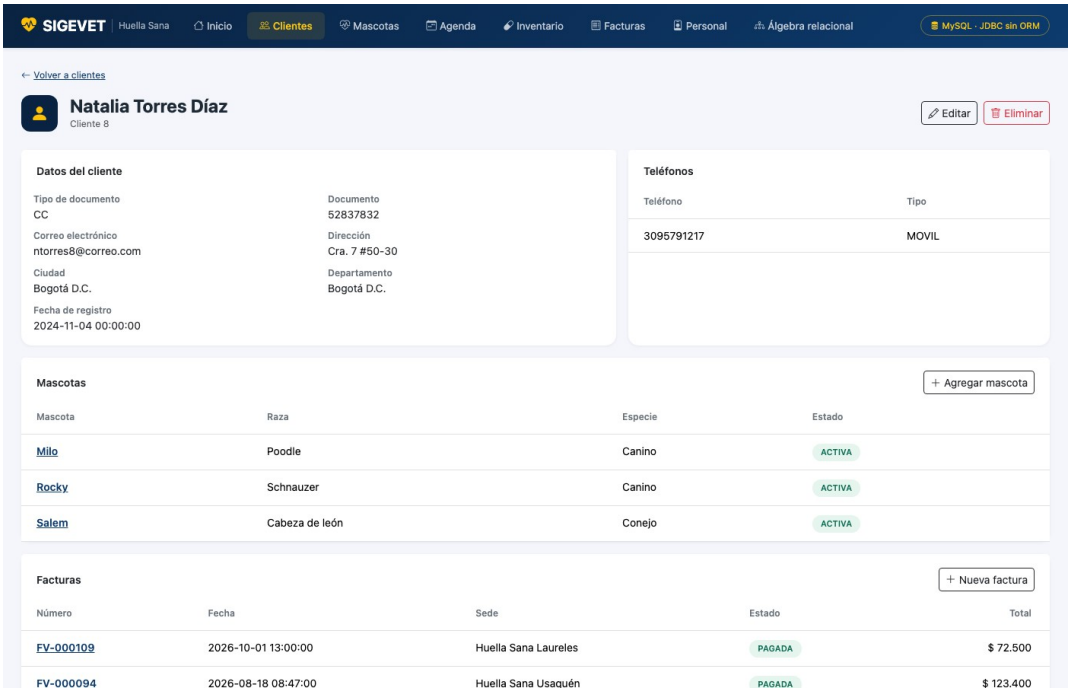
La última factura demuestra la transacción: el encabezado ya se había insertado cuando la línea del medicamento falló, y el ROLLBACK lo deshizo, así que no quedó una factura a medias.



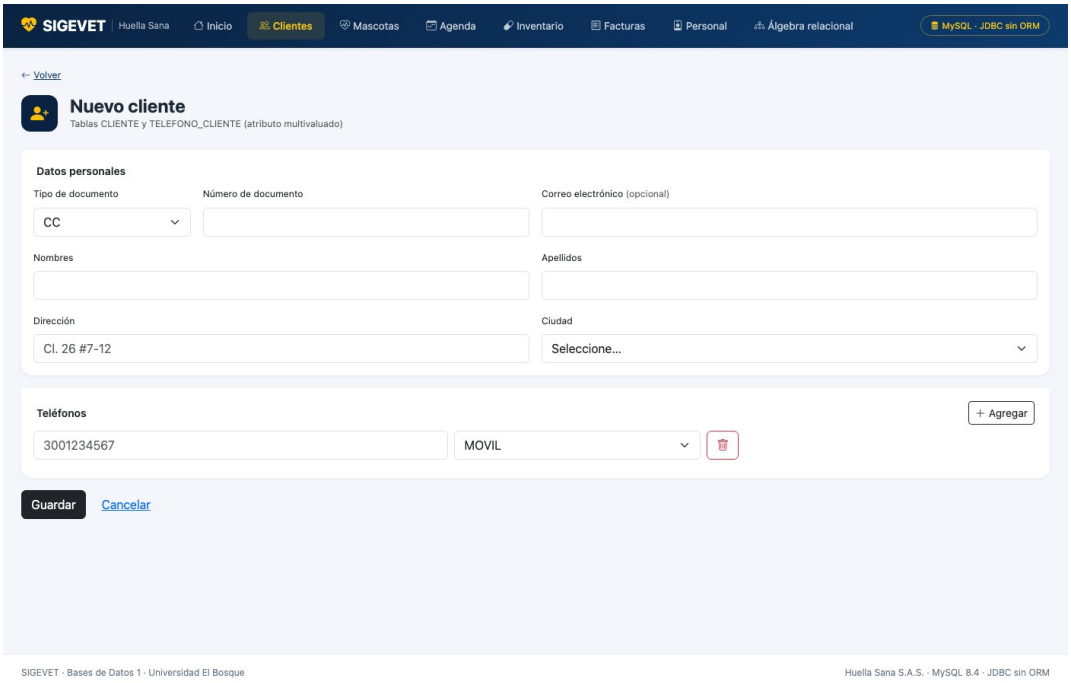
Inicio de sesión del personal con un documento incorrecto



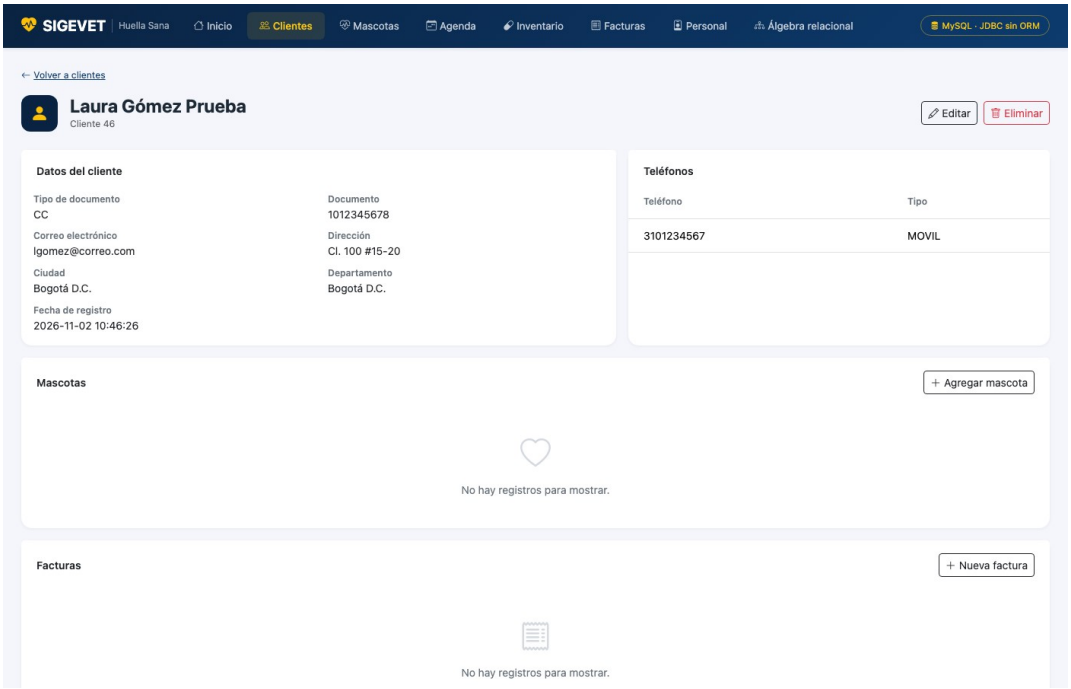
Página de inicio con los indicadores de la red



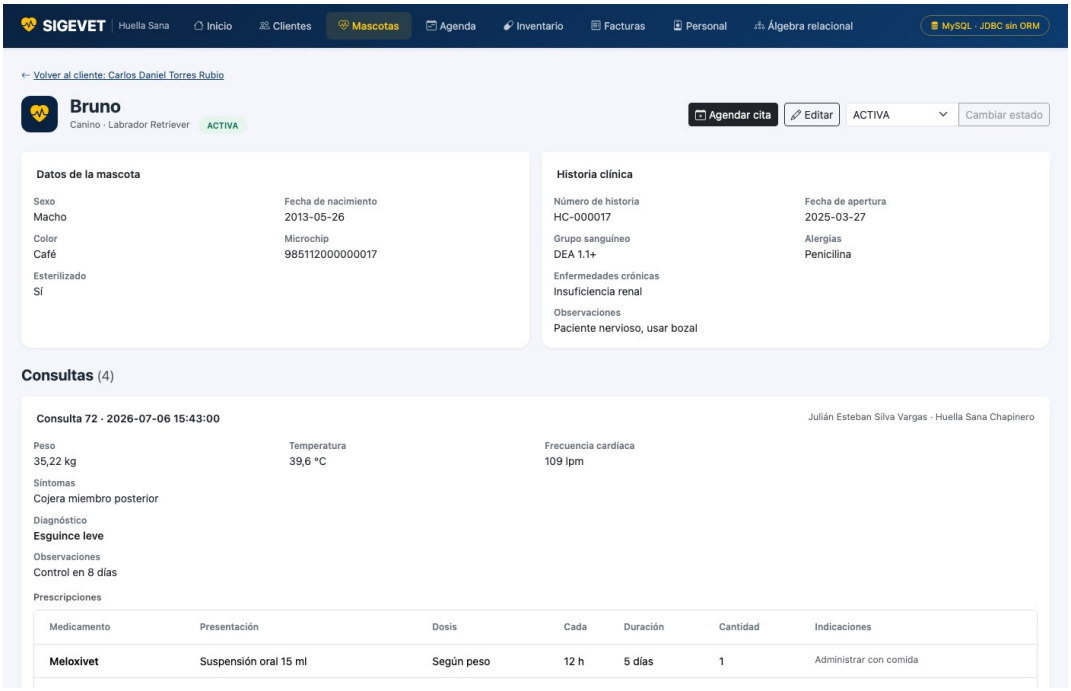
Detalle de un cliente con teléfonos, mascotas y facturas



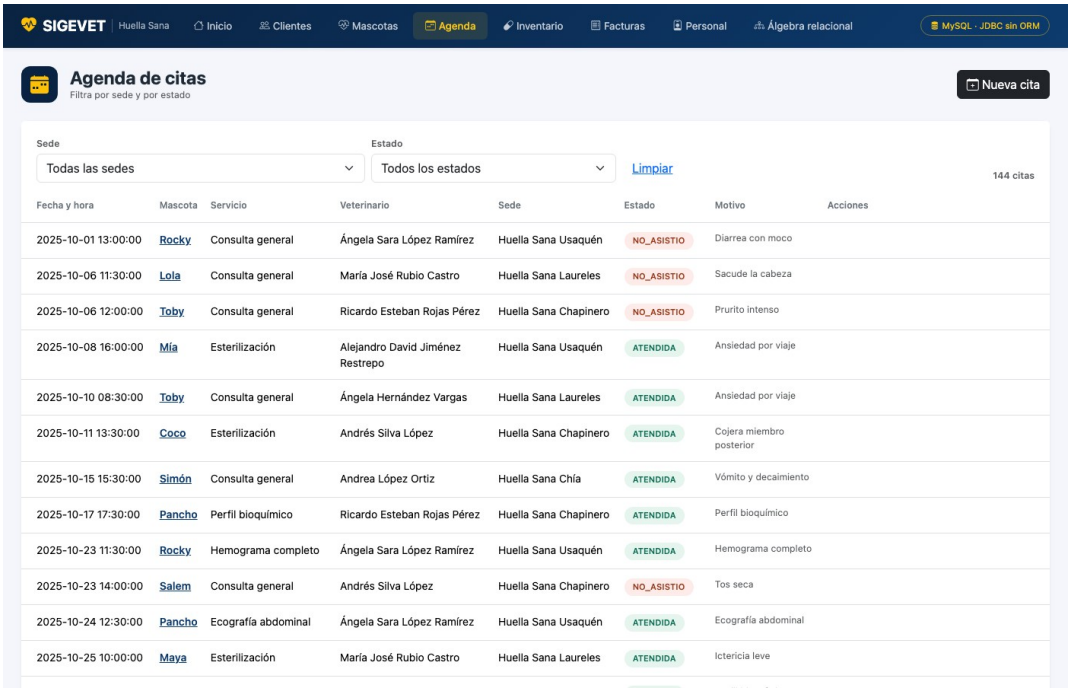
Formulario para crear un cliente



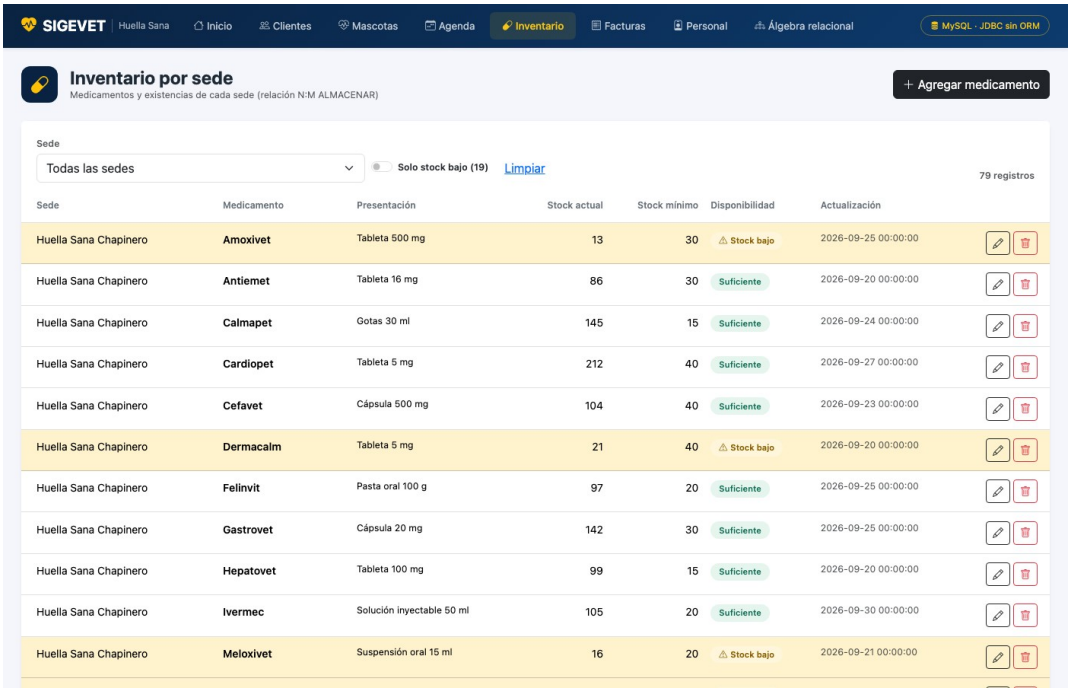
Cliente creado desde la aplicación



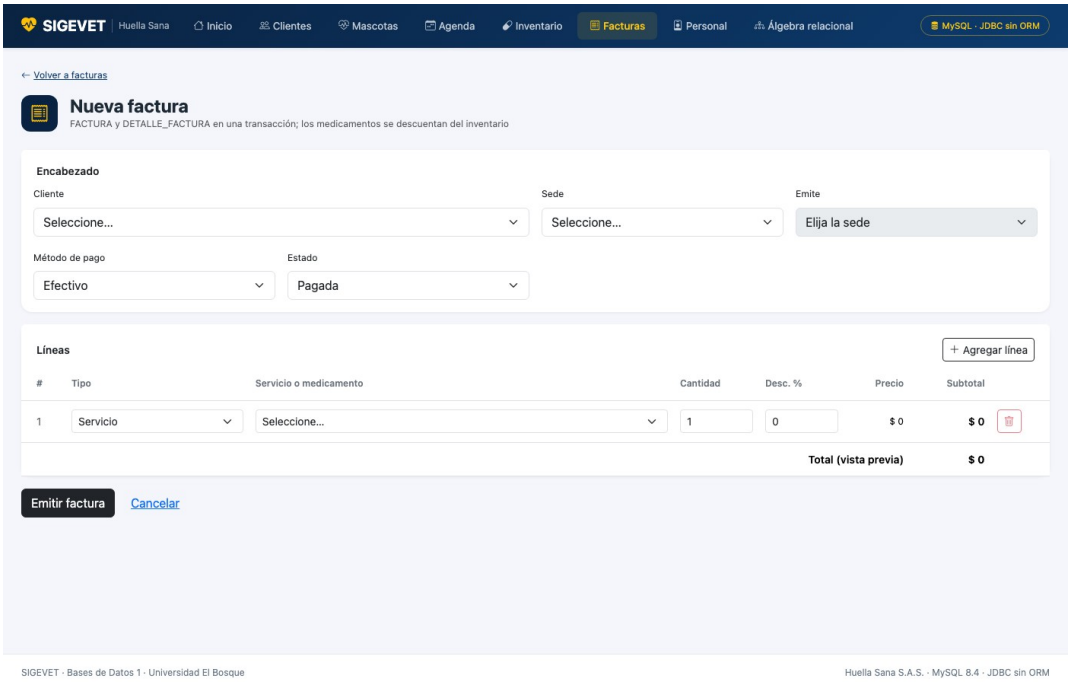
Historia clínica de una mascota (consultas y prescripciones)



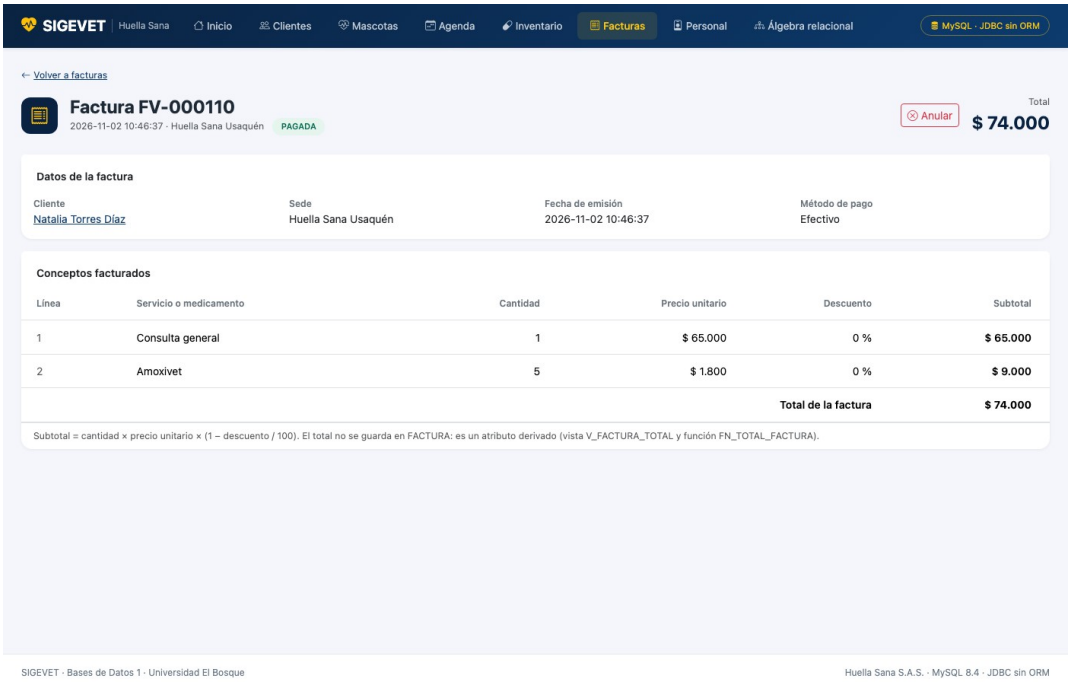
Agenda de citas



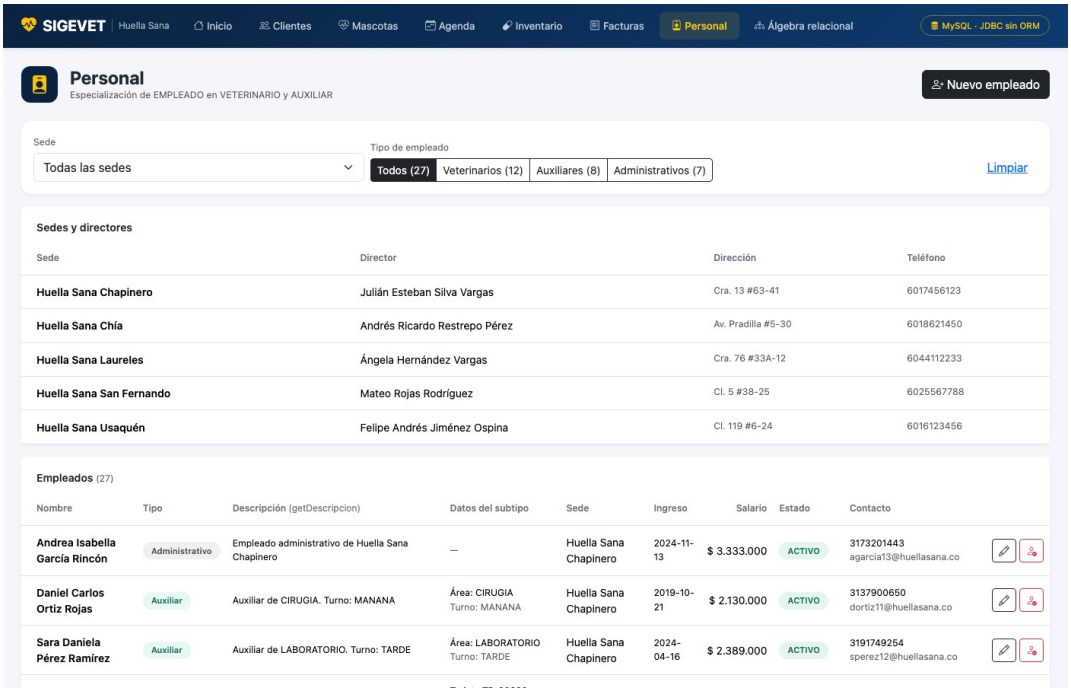
Inventario por sede con alertas de stock



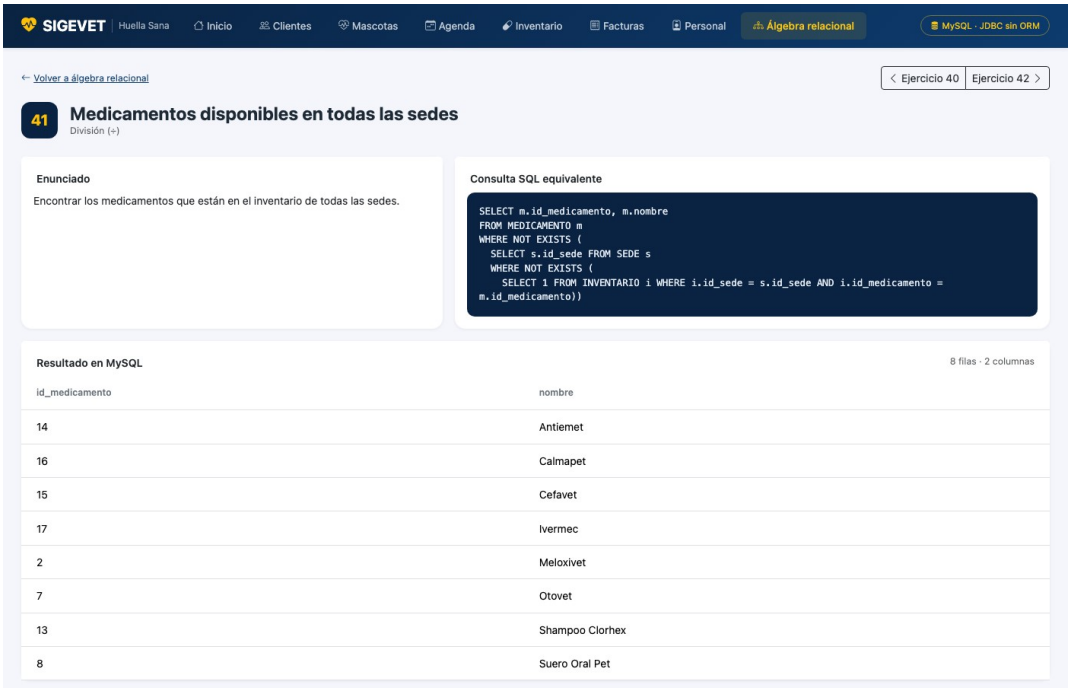
Formulario de una factura nueva con sus líneas



Factura creada desde la aplicación



Personal: cada tipo de empleado con su propia descripción (polimorfismo)



Ejercicio 41 de álgebra (división) ejecutado en vivo